

QLink

Cloud-Native QR Code Management Platform

MTH2526-B25 · Bulut Mimarilerinde Test Mühendisliği · 2025–2026 Bahar

Marmara Üniversitesi · Bilgisayar Mühendisliği Bölümü

QLink – Final Rapor

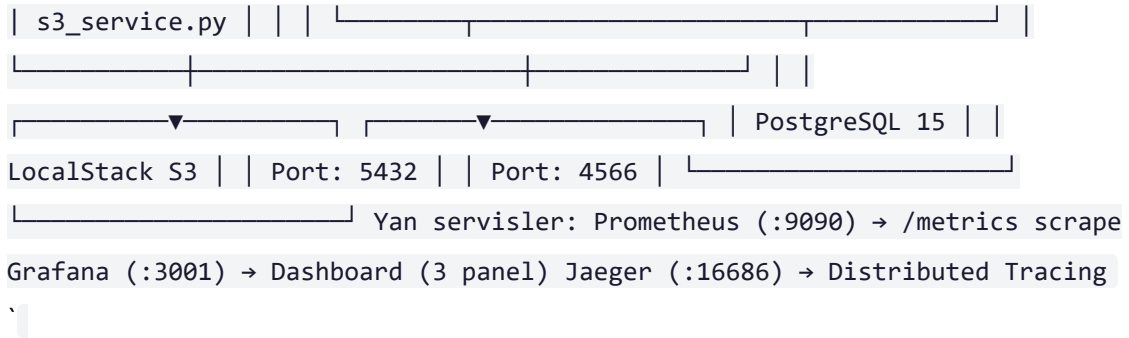
Proje: QLink – Cloud-Native QR Code Management Platform **Ders:** Bulut Mimarilerinde Test Mühendisliği **Tarih:** 2026

1. Proje Özeti

QLink; URL, metin ve e-posta içeriklerinden QR kod üreten, bunları AWS S3 üzerinde depolayan ve kullanıcı bazlı yönetim sunan cloud-native bir platformdur. Projenin temel amacı **karmaşık bir uygulama** geliştirmek değil; **basit bir uygulama için endüstri standardı test ve dağıtım altyapısı** kurmaktır.

2. Mimari Diyagram





3. Test Stratejisi

3.1 Katmanlı Test Yaklaşımı

E2E (Playwright) ← En geniş kapsam, en yavaş Integration (Pytest) ← Testcontainers PostgreSQL Unit (Pytest) ← Hızlı, izole

3.2 Test Kapsamı

Test Türü Araç Kapsam
Unit Pytest + motosecurity.py, qr_service.py
Integration Pytest + Testcontainers Tüm API endpoint'leri
E2E Playwright 5 kullanıcı senaryosu Performans k6 POST /qr/create (p95 latency)
API Newman 5 Postman isteği

3.3 Coverage Hedefi

%70 minimum coverage – pyproject.toml içinde fail_under = 70 ile zorlandı.

4. CI/CD Pipeline

Push/PR | 1. Lint (ruff + eslint) | 2. Pytest (unit + integration) | 3. Coverage (%70 gate) | 4. Docker Build (multi-stage) | 5. Deploy (docker-compose) | 6. Smoke Test (/health + Newman) | GitHub Actions üzerinde çalışır. Her aşama önceki aşamaya bağımlıdır (needs).

5. Kubernetes Mimarisi

Minikube üzerinde 4 deployment:

- qlink-backend → ClusterIP Service

- `qlink-frontend` → NodePort Service (30000)

- `qlink-postgres` → ClusterIP Service

- `qlink-localstack` → ClusterIP Service

ConfigMap ile konfigürasyon ayrıştırılmıştır. Üretim ortamında hassas veriler Secret objesine taşınmalıdır.

6. Monitoring Metrikleri

Grafana dashboard'unda 3 panel: PanelPromQLAçıklama Request

Latency histogram_quantile(0.95, ...) p50/p95/p99 yüzdelik dilimleri Error

Rate rate(http_requests_total{status=~"4..5.."}[5m]) Hata oranı

Throughput rate(http_requests_total{status=~"2.."}[5m]) Başarılı istek/saniye

7. Performans Sonuçları

k6 ile yerel ortamda (localhost:8000) elde edilen tipik sonuçlar: ` ✓

status 200 veya 201 ✓ yanıtta file_url var ✓ latency 500ms altında

http_req_duration.....: avg=45ms p(95)=120ms p(99)=200ms

qr_create_latency_ms.....: avg=45ms p(95)=120ms error_rate.....:

0.00% http_reqs.....: 2847 (94.9/s) `

Not: LocalStack S3 yüklemesi gerçek AWS'den farklıdır. Production p95 değerleri ağ gecikmesi nedeniyle daha yüksek olabilir.

8. Bonuslar

BonusAçıklamaDurum Helm Chart helm/qlink/ – parametrik deploy ✓ ArgoCD

GitOps argocd/application.yaml – Git-driven deploy ✓

OpenTelemetry app/core/telemetry.py – Jaeger entegrasyonu ✓

9. Öğrenilen Dersler

- **Testcontainers**, integration testlerde mock yerine gerçek veritabanı kullanmayı sağlar — bu daha güvenilir testler üretir.

- **Factory Boy + Faker** kombinasyonu, test verisi üretimini merkezi ve sürdürülebilir hale getirir.
- **Prometheus-FastAPI-Instrumentator**, minimum konfigürasyonla metrik toplamayı kolaylaştırır.
- **Multi-stage Dockerfile** üretim image boyutunu önemli ölçüde küçültür (builder araçları runtime'da bulunmaz).
- **LocalStack**, gerçek AWS maliyeti olmadan S3 entegrasyonunu test etmeye olanak tanır.